

RTL Design Sherpa

RAPIDS beats: the AXIS4 Side Stream Interfaces, Stimulus, Observers -- Version 1.0

September 2026

Table of Contents

1 Document Information....8
1.1 Revision History....8
1.2 How to read this book....9
2 Why rapids Is Not STREAM....10
2.1 Two engines, two stream ports....10
2.1.1 Figure 1.1: Two harnesses, one methodology....10
2.2 What that changes in the harness....10
2.3 Two directions, measured separately....12
3 The AXIS4 Interfaces....13
3.0.1 Figure 2.1: The two links and what stands on them....13
3.1 The wires....13
3.2 Sink ingress: a beat becomes a write burst....16
3.2.1 Figure 2.2: Sink ingress....16
3.3 Source egress....17
3.4 The AXIS monitor-lites....17
4 Stream Stimulus and Checking....19
4.1 The generator: axis4_master_injector....19
4.2 The checker: axis4_slave_pattern_check....20
4.3 The golden model and PASS....20
4.4 GO....21
5 Measuring a Stream....22
5.0.1 Figure 4.1: Meters, windows and observers on the stream side....22

5.1 The AXIS meters....22
5.2 The ingress window is its own....23
5.3 What the report reads from them....24
5.4 The AXIS observer....24

List of Figures

Figure 1.1: Two harnesses, one methodology.....10
Figure 2.1: The two links and what stands on them.....13
Figure 2.2: Sink ingress.....16
Figure 4.1: Meters, windows and observers on the stream side.....22

List of Tables

Table 1: Table 0.1: Document information....8
Table 2: Table 0.2: Revision history....8
Table 3: Table 1.1: What the streams change....10
Table 4: Table 2.1: The AXIS4 signals as built....13
Table 5: Table 3.1: Generator configuration....19
Table 6: Table 4.1: The cycle classification, shared with the AXI meters....22
Table 7: Table 4.2: The four columns....24
Table 8: Table 4.3: The AXIS tap's packet vocabulary....24

List of Waveforms

Waveform 4.1: The ingress window, before and after.....23

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

1 Document Information

Table 0.1: Document information

Field	Value
Title	RAPIDS beats on the Genesys 2 – The AXIS4 Side
Version	1.0
Date	2026-09-29
Scope	The stream side of the rapids-beats characterization system: the two AXIS4 links, their stimulus and checking, their meters and observers, and what differs from STREAM's memory-to-memory harness
Companion	`rapids_fpga_system` (board, harness, build variants, flow)
Not in scope	The RAPIDS core architecture (HAS/MAS); the measured results (`reports/perf/README.md`)
Status	Current; written against the tree at the BUG-009 close (commit 37455c992)

1.1 Revision History

Table 0.2: Revision history

Version	Date	Change
1.0	2026-09-29	First edition. Written after the AXIS monitor-lites (rapids TASK-015), the ingress window rework (ISSUE-001), the interleaved generator schedule (TASK-018), the 256-bit design point (report v2.0) and the

Version	Date	Change
		ingress findings of BUG-009.

1.2 How to read this book

Chapter 1 says why the rapids system is a different build from STREAM's and what that changed in the harness. Chapter 2 is the DUT's two stream ports as the harness sees them. Chapter 3 is the stream stimulus and checking, which STREAM has no equivalent of. Chapter 4 is how a stream is measured: the AXIS meters, why the ingress window is its own, and what the AXIS observers add. Every diagram is a Graphviz source under `assets/graphviz/`.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

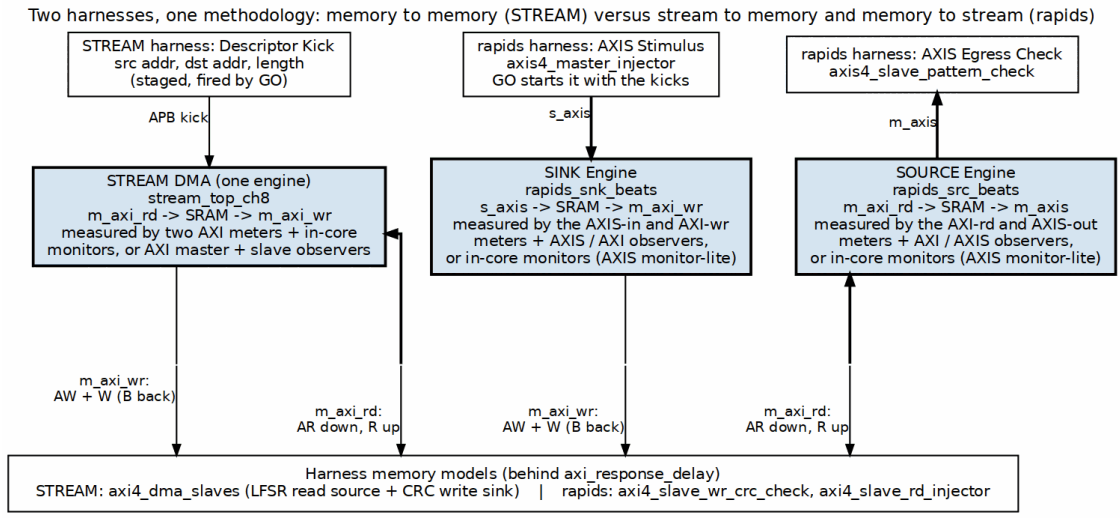
2 Why rapids Is Not STREAM

2.1 Two engines, two stream ports

STREAM moves memory to memory: one DMA, one descriptor with a source and a destination address, two AXI4 data masters. Its harness is therefore a memory model on each side, an LFSR read source behind the read master and a CRC write sink behind the write master, and everything measured is on an AXI4 bus.

RAPIDS beats is two engines that share nothing but an APB and a MonBus egress. The SINK takes an AXI4-Stream in and writes memory; the SOURCE reads memory and puts an AXI4-Stream out. Each descriptor has one memory address and one stream. So the rapids harness needs what STREAM's never did: a stream producer that the sink can be fed from, and a stream consumer that can check what the source emits, both deterministic, both seeded, both on chip. Half of the DUT's data ports are streams, and half of the characterization is about them.

2.1.1 Figure 1.1: Two harnesses, one methodology



STREAM harness and rapids harness side by side

Source: 04_rapids_vs_stream.dot

2.2 What that changes in the harness

Table 1.1: What the streams change

Aspect	STREAM	rapids
Data ports on the DUT	two AXI4 masters	two AXI4 masters, one AXIS slave, one AXIS

Aspect	STREAM	rapids
		master
Directions under test	one, memory to memory	two, independent: stream to memory (SINK) and memory to stream (SOURCE)
What starts the data	the descriptor kick	GO starts the AXIS generator and fires the kicks in the same few cycles; the sink cannot move until its stream arrives
Channel on the wire	the AXI id	`tid`; the generator sets it, the sink routes on it, the checker demuxes on it, the observer attributes on it
Packet boundary	none; the descriptor length is the unit	`tlast`, placed by the generator every `GEN_BPP` beats and carried by the sink into its fill interface
Backpressure knob	the write slave's readiness	the checker's `chk_ready_en` on the egress stream, and the DUT's own `s_axis_tready` on ingress
Meters	two, on the AXI masters	four: the two AXI meters plus an AXIS meter on each stream, which also count exact bytes and packets
Ingress window	the shared window	the stream's own window, opened by the first offered beat and

Aspect	STREAM	rapids
		closed by the last accepted one
Observers	AXI master and AXI slave observers	the AXI master observer and a two-port AXIS observer with per-`tid` attribution
In-core monitors	AXI and descriptor monitors	the same, plus an AXIS monitor-lite in each half

Everything else is the same methodology, and deliberately so: the same `axi_bus_meter`, the same `axi_response_delay` latency model, the same observer register map, the same `uart_axil_bridge`, the same stage-all-then-GO launch, the same golden-CRC pass criterion. The host code and the report generator run on both DMAs. What this book covers is the part that could not be shared.

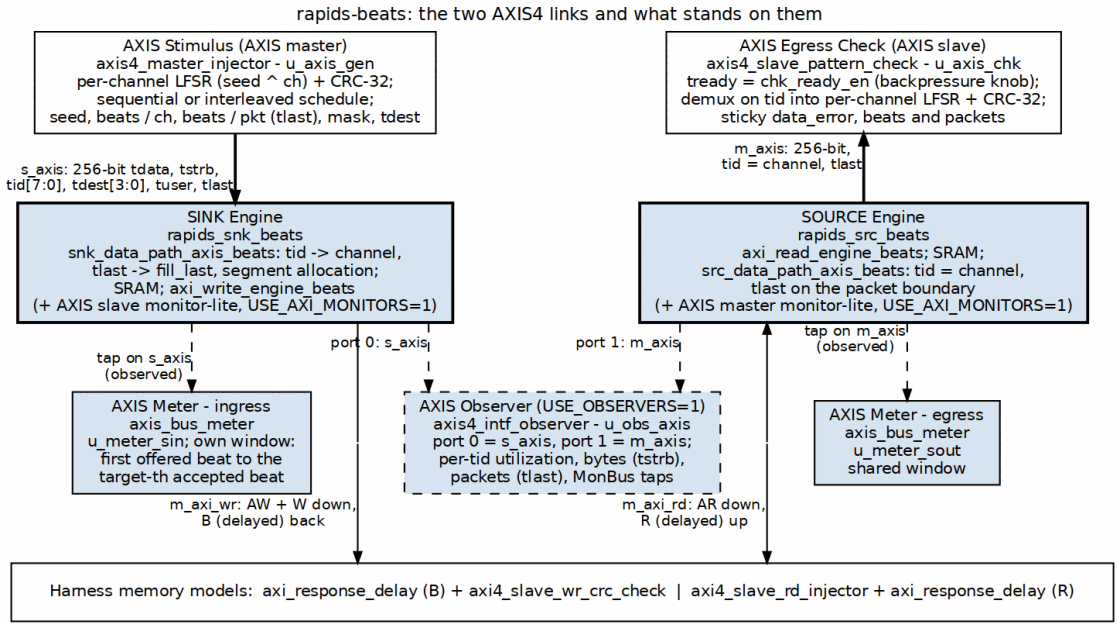
2.3 Two directions, measured separately

Because the two engines are independent, the campaign runs each direction as its own self-check. The sink self-check is the generator into `s_axis`, through the SINK, out of `m_axi_wr` into the CRC sink; the source self-check is the LFSR memory behind `m_axi_rd`, through the SOURCE, out of `m_axis` into the checker. A configuration row carries both verdicts and both sets of meters, and `--sink-only` or `--source-only` runs one. When a knob affects only one side (an AxLEN change reaches both engines; `chk_ready_en` reaches only the source), the report says which column moved.

The two directions also do not cost the same. The SOURCE pre-allocates SRAM for every read burst, so under memory latency its in-flight reads are bounded by the per-channel buffer and the read column is the one that falls first. The SINK frees SRAM on the write handshake, not on the write response, so its window is the outstanding count times the burst and memory latency costs it nothing until that window is exhausted. Those are report findings (section 7.7); the point here is that a stream-fed engine and a stream-producing engine are different machines, and the harness has to be able to tell them apart.

3 The AXIS4 Interfaces

3.0.1 Figure 2.1: The two links and what stands on them



The two AXIS4 links

Source: 01_two_axis_links.dot

3.1 The wires

Both links are full AXI4-Stream with the sideband the DUT is parameterized for. `DATA_WIDTH` is the one width RAPIDS has, so the stream is as wide as the memory bus: 256 bits and a 32-bit `tstrb` since the v2.0 design point.

Table 2.1: The AXIS4 signals as built

Signal	Sink ingress `s_axis_*`	Source egress `m_axis_*`	Width	Meaning
`tvalid`, `tready`	in, out	out, in	1	the handshake; every meter and observer

Signal	Sink ingress `s_axis_*`	Source egress `m_axis_*`	Width	Meaning
				classifies a cycle by these two bits
`tdata`	in	out	`DATA_WIDTH`	payload
`tstrb`	in	out	`DATA_WIDTH/8`	byte enables; the AXIS meter's byte count is the popcount of this on productive beats
`tlast`	in	out	1	packet boundary; becomes `fill_last` inside the sink, is generated on the packet bounda

Signal	Sink ingress `s_axis_*`	Source egress `m_axis_*`	Width	Meaning
				ry by the source
`tid`	in	out	`AXIS_ID_WIDTH` = 8	the channel: the low bits select the SRAM channel on ingress and identify the producing channel on egress
`tdest`	in	out	`AXIS_DEST_WIDTH` = 4	routing sideband, carried through; the harness sets it from `GEN_TDEST`
`tuser`	in	out	`AXIS_USER_WIDTH` = 1	user sideband, carried

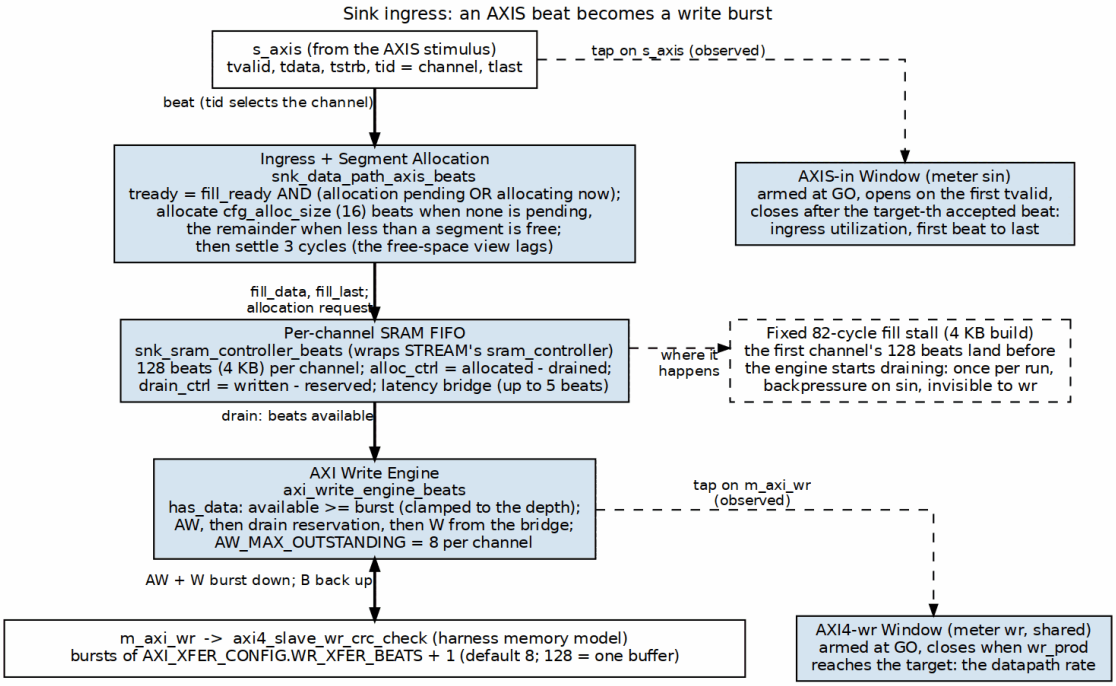
Signal	Sink ingress `s_axis_*`	Source egress `m_axis_*`	Width	Meaning

through

The DUT is the AXIS slave on ingress and the AXIS master on egress. That is why the harness has an AXIS *master* generator and an AXIS *slave* checker, and why the in-core monitor-lites are an AXIS slave monitor in the sink half and an AXIS master monitor in the source half.

3.2 Sink ingress: a beat becomes a write burst

3.2.1 Figure 2.2: Sink ingress



Sink ingress: allocation, SRAM, write engine, and the two windows

Source: 02_sink_ingress.dot

`snk_data_path_axis_beats` turns the stream into the fill interface of the sink data path. `tid` selects the channel, the beat goes straight through as `fill_data`, `tlast` becomes `fill_last`. What makes ingress more than a wire is SRAM allocation: the sink reserves space before it accepts data, in segments of `cfg_alloc_size` beats (16, `AXI_XFER_CONFIG.ALLOC_SIZE`), and `s_axis_tready` is high only while the beat's channel holds an allocation or is allocating this cycle, and the FIFO can take the beat.

Three properties of that allocation were established this month and are worth knowing when reading an ingress number:

- A packet that ends mid-segment leaves the rest of its segment allocated for the channel's next packet. The next packet consumes it before it allocates anything new, so per-channel accounting stays consistent across packets and runs.
- When less than a segment is free, the ingress allocates the remainder, so the buffer can always be filled to the top. Without that, a write burst equal to the buffer depth could never be satisfied after a short packet (rapids BUG-009, second finding).
- After any allocation a channel settles three cycles before allocating again, because the free-space view it reads lags an allocation by that much. A one- or two-beat remainder consumed before the view moved once let the allocator over-commit (BUG-009, third finding).

Backpressure on `s_axis_tready` therefore means one of: the channel's buffer is full, no space is allocated yet, or the pipeline is stalled. On the 4 KB build the first of those happens once per run in a fixed way: the generator lands the first channel's 128 beats before the write engine has started draining, and the ingress waits 82 cycles. It is the same 82 at every transfer size, a startup cost and not a throughput one, and the 16 KB buffers of the earlier design point absorbed it entirely.

3.3 Source egress

`src_data_path_axis_beats` is the mirror: the read engine fills the per-channel SRAM from `m_axi_rd`, and the egress side drains it onto `m_axis` with `tid` set to the channel and `tlast` on the packet boundary. The source's throughput under memory latency is bounded by that SRAM, since the read engine allocates the whole burst before issuing the AR; the egress link itself, being a master into an always-ready checker, is never the limit unless the host asks for backpressure.

3.4 The AXIS monitor-lites

With `USE_AXI_MONITORS=1` each half carries an AXIS monitor-lite on its stream port (rapids TASK-015, "Option B"): an AXIS slave monitor in `rapids_snk_beats` on `s_axis`, an AXIS master monitor in `rapids_src_beats` on `m_axis`, each a third client of its half's MonBus arbiter beside the descriptor and scheduler emitters. They keep the one-monitor-stream-per-half contract and let the half-level tests see the packets. Their filtering is configured through the MonBus group's AXIS slot, and they carry agent ids next to the AXI descriptor monitor's. The monitors-in build of the companion book is where they are built and measured; the bare and observer builds leave them out, which is why those builds have no in-core view of the streams and the AXIS observer of Chapter 4 exists.

4 Stream Stimulus and Checking

STREAM's harness needs memory models. The rapids harness needs those and two stream endpoints, and the endpoints have to be as deterministic as the memory models so that a CRC can be predicted in software. Both are `rtl/amba/shared` blocks, built for this harness and reused by STREAM's generator since the interleave work.

4.1 The generator: `axis4_master_injector`

An AXIS master with no address channel. Per channel it runs an independent LFSR seeded with `seed ^ channel` and a CRC-32 over what it emits, the same LFSR and CRC as `axi4_slave_rd_injector`, so a stream integrity check and a memory integrity check produce comparable numbers. The LFSR and the CRC advance only on accepted beats: beat N of channel C is a function of `(seed ^ C, N)` regardless of how `tready` stalled it, which is what makes the pattern predictable under backpressure and under the interleaved schedule.

Table 3.1: Generator configuration

Register (harness CSR)	Field	Effect
`GEN_CTRL`	`cfg_gen_start`	arm one run; the actual start comes from GO
`GEN_LFSR_SEED`	32 bits	the base seed; `0xDEADBEEF` is the campaign default, `--base-seed` overrides it for the sink half
`GEN_NUM_BEATS`	32 bits	beats per channel
`GEN_BEATS_PPKT`	32 bits	`tlast` cadence; 0 means one packet per channel
`GEN_CH_MASK`	one bit per	active

Register (harness CSR)	Field	Effect
	channel	channels; 0 means all
GEN_TDEST	AXIS_DEST_WIDTH	the tdest value on every beat
GEN_MODE	INTERLEAVE	the channel schedule, below

Two schedules. **Sequential** finishes one channel's beats, then the next: each channel's stream is contiguous, and only one sink channel holds data at a time. **Interleaved** (GEN_MODE.INTERLEAVE, --interleave, rapids TASK-018) hands the bus to the next active channel after every accepted beat, so tid changes every beat and every sink channel holds data at once. A channel's LFSR and CRC still advance only on its own beats, so the per-channel data, CRCs and packet boundaries are identical under both schedules and the golden model does not care which ran. The difference is what the sink column measures: one channel's window under the sequential schedule, the DUT's aggregate window under the interleaved one (report Tables 7.5 and 7.5b).

The generator reports per-channel expected CRCs and beat counts, and an aggregate total, as corroboration. PASS never rests on them.

4.2 The checker: axis4_slave_pattern_check

An AXIS slave and a pure sink. Its tready is the ready_en input, which the harness exposes as CHK_CTRL.chk_ready_en: the campaign's source-backpressure knob (--backpressure, --suite-bp) is this bit held low in a pattern, and it reaches only the source half since the sink's egress is AXI. Incoming beats are demuxed by tid into the matching channel context, compared beat by beat against the locally regenerated pattern with the same seed ^ channel LFSR, and accumulated into a per-channel CRC-32. A mismatch sets a sticky data_error; the aggregate beat and packet counters (CHK_BEATS_TOTAL, PKT_COUNT) are what the host polls for completion of the source half.

Because the check is local and per beat, it is independent of upstream stalls and of cross-channel interleave: the source may emit its channels in any order the scheduler chooses.

4.3 The golden model and PASS

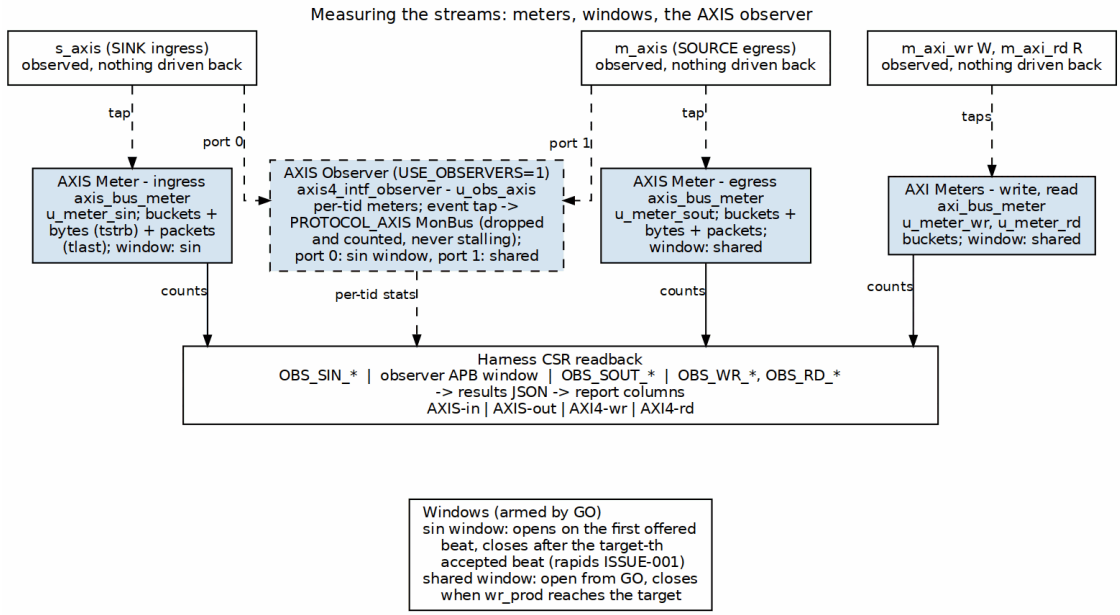
host/rapids_char_golden.py runs the same LFSR and CRC in software. For the sink half, PASS is the write CRC the memory-side checker computed equal to the model's; for the source half, it is the read CRC the memory-side generator produced and the checker's egress CRC both equal to the model's. The generator's own expected CRC and the checker's own count are read as corroboration and logged, never as the verdict.

4.4 GO

The stream is the one stimulus STREAM's harness does not have to start. In rapids the kick sequencer's GO does three things within a few `aclk` cycles: arms the meter windows, fires every staged descriptor kick, and starts the generator. Nothing on the sink side moves until the generator's first beat is offered, which is why the ingress window is defined by that beat and not by GO (Chapter 4).

5 Measuring a Stream

5.0.1 Figure 4.1: Meters, windows and observers on the stream side



Meters, windows, observers

Source: 03_axis_measurement.dot

5.1 The AXIS meters

`axis_bus_meter` is the AXIS analogue of `axi_bus_meter`, one instance per stream, snooping `tvalid`, `tready`, `tlast`, `tstrb` and `tid` and driving nothing. Every cycle of its window goes into one bucket:

Table 4.1: The cycle classification, shared with the AXI meters

`tvalid`	`tready`	Bucket	Meaning
1	1	productive	a beat transferred
1	0	backpressure	the master wants to send, the slave stalls
0	1	starvation	the slave is ready, the master is not

tvalid	tready	Bucket	Meaning
			producing
0	0	idle	both quiet

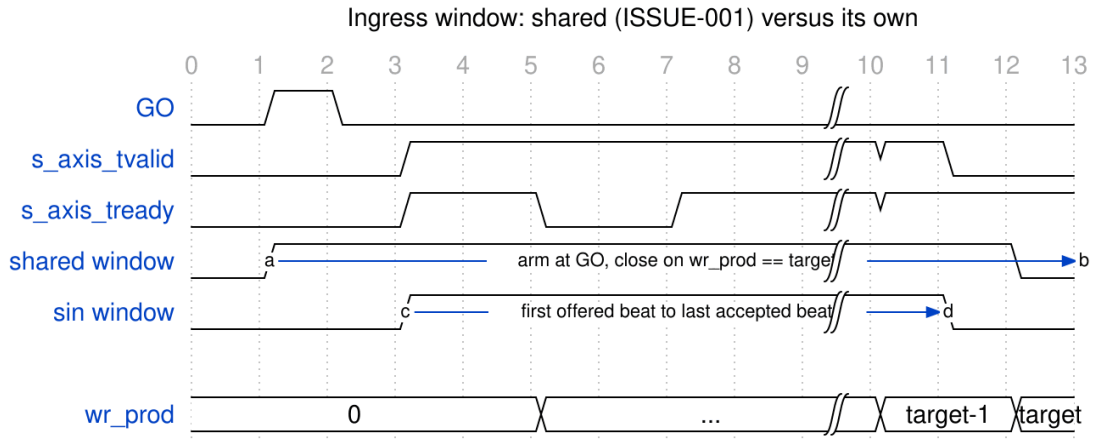
On top of the buckets the AXIS meter counts what an AXI meter cannot: the exact number of payload bytes moved, as the popcount of `tstrb` on each productive beat, and packets, as `tlast` handshakes. Those two are window-independent, so a throughput computed from bytes over busy time is robust to the idle and backpressure padding that distorts a pure cycle utilization when a window is held open too long. The host reports both: utilization from the buckets and the byte-derived figure beside it.

5.2 The ingress window is its own

Every AXI-side meter and the egress meter share one window: armed at GO, closed the cycle after the write beat count reaches the staged target. For the ingress stream that definition measured the wrong thing. The stream starts at the kick, before the sink's write side is busy, and after the last beat is accepted the ingress sits ready and idle while the sink drains to memory, about 200 cycles on the board regardless of transfer size. With the shared window, ingress starvation was a constant, ingress utilization was the minimum of the direction on nearly every row, and the reported sink throughput was launch latency, not datapath rate (rapids ISSUE-001).

So `sin` has its own window. It arms at GO, opens on the first cycle the generator offers `tvalid` (a sink not yet ready then shows as backpressure, which is ingress behaviour), and closes the cycle after the target-th ingress beat is accepted: first beat to last beat, which is what ingress utilization means. The open is combinational so that first handshake is counted rather than lost to a register delay; the close falls back to the shared window when no target is staged. The AXIS observer's port 0 uses the same window, so the bare meter and the observer agree to the beat.

5.2.0.1 Waveform 4.1: The ingress window, before and after



Ingress window: shared versus its own

Source: 01_ingress_window.json

5.3 What the report reads from them

Table 4.2: The four columns

Column	Meter	Window
AXIS-in (`sin`)	`axis_bus_meter` on `s_axis`	its own: first offered beat to last accepted beat
AXI4-wr (`wr`)	`axi_bus_meter` on `m_axi_wr` W	shared
AXI4-rd (`rd`)	`axi_bus_meter` on `m_axi_rd` R	shared
AXIS-out (`sout`)	`axis_bus_meter` on `m_axis`	shared

The one AXIS-specific residue in the headline rows is on `sin`: a single starvation cycle, the registered close landing one cycle after the last accepted beat, and on the 4 KB build the fixed 82-cycle fill stall as backpressure. Everything else on the stream side reads at line rate once transfers are descriptor-sized.

5.4 The AXIS observer

`axis4_intf_observer` (`USE_OBSERVERS=1`) is the AXIS sibling of the AXI interface observers: the same APB window layout, the same register map, the same MonBus egress and telemetry readback, so one host decoder serves all three and the report generator runs unchanged on STREAM's harness and on this one. The harness instantiates it with two ports, port 0 on `s_axis` and port 1 on `m_axis`, every pin an input, both halves of the handshake included: attaching it cannot change the stream it measures.

Per port it builds an `axis_bus_meter` of its own with per-`tid` channel attribution, read through `OBS_STAT_SEL` and `OBS_STAT_DATA`, and an event tap that emits `PROTOCOL_AXIS` MonBus packets. There is no AXIS monitor in the AMBA monitor library, so the tap was written in the monitor-lite's discipline: no CAM, no timer pool, stamps rather than counters, and an event the MonBus cannot take is dropped and counted (`OBS_STICKY.TAP_BLOCKED`) rather than allowed to stall anything.

Table 4.3: The AXIS tap's packet vocabulary

Class	Event	Fires when
Error	`VALID_TIMING`	`tvalid` dropped before `tready` accepted

Class	Event	Fires when
Error	STRB_INVALID	a beat handshook with tstrb all zero
Timeout	HANDSHAKE, PACKET	a stall, or an in-packet gap, reached MON_TIMEOUT
Completion	STREAM_END	tlast handshook
Credit	BACKPRESSURE	a stall reached MON_LATENCY cycles (there is no Threshold class for AXIS)
Channel	ID_CHANGE, DEST_CHANGE	tid or tdest changed inside a packet
Stream	START, PAUSE, RESUME	first beat of a packet; tvalid dropped or returned mid-packet

Runtime gating reuses MON_CTRL bit for bit with the AXI observers, the address-range checker is inert because a stream has no address, and OBS_ENABLE_MON_TAPS decides at build time whether the tap logic exists at all; the meters and the histograms count without it.

What the observers add over the bare meters, measured: the same utilization numbers to the beat, plus exact bytes and packets per port with per-tid attribution, the tap's own packet count as a cross-check, and a sticky bit that says when a number undercounts (report section 7.6). What they cost: the observers build closes at less margin than the bare build, and that difference is the instrument's price, not the datapath's.